Qwen3-Coder-Next Tier 3 Part 1 Detailed Implementation Specification

Foundation

Engineering specification for the first buildable runtime layer of the Qwen3-Coder-Next autonomous software engineering system.

Document type	Detailed implementation specification
Target part	Part 1 - Foundation
Audience	Principal engineer / implementation engineer
Assumption	Architecture and roadmap already finalized

Part 1 at a glance

Foundation is the smallest complete runtime layer that can boot, load configuration, create a run context, persist state, log activity, invoke the base model, and return a deterministic response.
It is intentionally narrow. Anything that requires planning, memory retrieval, tool use, testing, or evaluation belongs to later parts.

1. Purpose

Part 1 exists to establish a stable execution substrate for every later capability in the system. Before the agent can plan, research, test, evaluate, or recover from failure, it needs a runtime that boots cleanly, owns its state, and behaves the same way every time.

This part solves the unglamorous but fatal problem of foundation drift. Without a disciplined runtime core, later components will inherit inconsistent config loading, ad hoc state handling, opaque logs, and no reliable place to store artifacts or checkpoints.

Future parts that depend on this foundation include the Filesystem + Local Tooling layer, the Planning layer, Memory, Testing and Review, Evaluation, Failure Recovery, Repository Intelligence, Context Compression, Benchmark Harness, and Deployment with Human Approval.

Why this matters

If the base runtime is unstable, every future subsystem becomes harder to debug, harder to verify, and harder to trust.
Part 1 is the contract that later parts build against. It is not the smartest part of the system; it is the part that makes intelligence operational.

2. Scope

Included in Part 1	Repository scaffold, runtime bootstrap, configuration loading, structured logging, task/session state model, message envelopes, minimal model gateway, artifact path conventions, orchestration shell, and smoke tests.
Excluded from Part 1	Planner logic, research retrieval, agent specialization, vector memory, testing loops, review policy, evaluator scoring, failure recovery strategy, repository graphing, benchmark harness, and deployment workflows.
Boundary rule	Part 1 may define interfaces for future systems,

	but it must not implement their intelligence or business logic.

3. System Overview

The foundation runtime is a thin control plane. Its job is to normalize input, create a run context, invoke the model through a stable interface, record state transitions, and emit a bounded response. Side effects such as logging and artifact writes are treated as first-class outputs, not as incidental debug noise.

```
User / CLI / API
  ↓
Runtime Entry Point
  ↓
Config Loader + Validator
  ↓
Session + Task State Initialization
  ↓
Minimal Orchestration Shell (LangGraph stub)
  ↓
Model Gateway
  ↓
Response Formatter
  ↓
User Output

Side channels:
 Logging / Tracing  ← every step
 State Store        ← checkpoints and metadata
 Artifact Manager   ← files, manifests, run records
```

The essential design decision is separation of concerns. The runtime coordinates, but it does not yet reason deeply. The model speaks through a contract, but it does not directly own persistence or filesystem access. That separation prevents later parts from becoming tangled in the first week of implementation.

4. Internal Architecture

Part 1 should be organized around a small set of explicit runtime responsibilities. Each component is intentionally narrow so that later parts can extend the system without rewriting the core.

Component	Responsibility	Dependencies	Consumes / Produces
Runtime Entry Point	Starts the process, loads config, wires	CLI / env / config file	Consumes task request; produces run

	core services, and launches a single task run.		context
Config Loader	Resolves environment, defaults, and file-based settings into a validated config object.	Filesystem, environment variables	Consumes raw settings; produces typed config
Runtime Context	Owns immutable run metadata and shared references for the lifetime of a request.	Config, state store, logger, model gateway	Consumes init inputs; produces execution context
State Store	Persists task/session records and checkpoint metadata.	Filesystem or local DB	Consumes state mutations; produces reloadable state
Message Envelope Manager	Normalizes all messages into a stable internal schema.	Runtime context	Consumes user/system text; produces message records
Model Gateway	Provides a single interface for model invocation and response normalization.	Qwen3-Coder-Next endpoint / local runtime	Consumes message bundle; produces model response
Artifact Manager	Creates and indexes run artifacts, manifests, and checkpoints.	Filesystem path policy	Consumes outputs; produces artifact references
Logger / Tracer	Records structured events for boot, state changes, and errors.	Config and runtime context	Consumes events; produces structured logs
Orchestration Shell	Hosts the minimal LangGraph flow for the foundation stage.	Runtime context, model gateway	Consumes task request; produces final response

Required abstraction rule: every component communicates through declared data contracts. Direct calls into later-stage logic are forbidden, because that is how systems quietly turn into a pile of special cases with a README.

Conceptual interaction model

```
Runtime Entry
  ├─> Config Loader
  ├─> Logger
  ├─> State Store
  ├─> Artifact Manager
  └─> Orchestration Shell
          └─> Message Envelope
          └─> Model Gateway
          └─> Response Formatter
```

5. Project Structure

The repository should expose a predictable, boring structure. Boring is good here. Boring means future agents can find things, tests can locate modules, and engineers do not waste time asking where the runtime decided to hide its own brain.

```
src/qwen3_coder_next/
  bootstrap/       # process start, CLI entry, dependency wiring
  runtime/         # runtime context, task lifecycle, orchestration shell
  config/          # environment resolution, schema, validation
  contracts/       # request, response, state, event, artifact schemas
  state/           # session/task persistence and checkpoint logic
  logging/         # structured logging, tracing, correlation IDs
  adapters/        # model gateway and future external interfaces
  artifacts/       # run outputs, manifests, storage conventions
  prompts/         # base system prompts and prompt fragments
  utils/           # small shared helpers with no business logic
tests/
  unit/
  integration/
  smoke/
configs/
docs/
scripts/
```

Folder responsibilities should be explicit. Runtime code belongs in runtime. Contract definitions belong in contracts. Persistence logic belongs in state. If a module has to cross three boundaries to do one thing, it is probably already a future refactor in disguise.

6. Data Contracts

Part 1 must define the first stable language of the system. Every later part will speak through these contracts, so they need to be simple, explicit, and versionable.

6.1 Input structures

```
TaskRequest
  - request_id
  - session_id
  - user_goal
  - mode
  - constraints
  - metadata

RuntimeConfigInput
  - model_settings
  - storage_settings
  - logging_settings
  - safety_settings
```

6.2 Output structures

```
TaskResult
  - request_id
  - status
  - summary
  - response_text
  - artifact_refs
  - checkpoint_ref

ModelResponse
  - content
  - token_usage
  - latency_ms
  - raw_metadata
```

6.3 Task state

```
TaskState
  - task_id
  - session_id
  - lifecycle_status
  - current_step
  - active_message_ids
  - artifact_ids
  - last_error
  - timestamps
```

6.4 Message and artifact records

```
MessageRecord
```

```
  - message_id
  - role
  - content
  - parent_id
  - created_at
  - tags

ArtifactRecord
  - artifact_id
  - type
  - path
  - checksum
  - owner
  - created_at
```

6.5 Configuration format

```
RuntimeConfig
  - app_name
  - environment
  - model_endpoint
  - default_model
  - storage_root
  - log_level
  - checkpoint_policy
  - safety_flags
```

Design rule: contracts should be serializable without hidden runtime context. If a later agent cannot reconstruct the important state from the persisted record, the contract is too shallow.

7. State Management

State in Part 1 should be split by ownership and lifespan. This prevents the common failure mode where one monolithic object slowly accumulates every concern in the system and becomes impossible to reason about.

State layer	Owner	Persistence	Purpose
Process state	Runtime context	In memory	Active references for the current execution
Session state	State store	Persistent	Continuity across multiple requests in a single session
Task state	Orchestration shell	Persistent checkpoint	Track progress, step status, and terminal

			outcome
Artifact index	Artifact manager	Persistent	Record produced files and their lineage
Configuration snapshot	Bootstrapper	Persistent copy	Keep the exact config used for a run

State transitions should be append-heavy and overwrite-light. The runtime should record what changed, who changed it, and when. Later components such as memory, evaluation, and recovery will rely on that history to explain behavior and recover from bad runs.

```
Bootstrap
  -> load config
  -> create run_id
  -> create or load session
  -> initialize task state
  -> write initial checkpoint

Run
  -> append messages
  -> append events
  -> persist artifacts
  -> checkpoint after key transitions

Shutdown
  -> finalize result
  -> mark terminal state
  -> flush logs and checkpoint
```

8. Component Specifications

Component	Purpose	Inputs	Outputs	Failure conditions	Future integration points
Bootstrapper	Create a fully initialized runtime.	CLI args, env, config file	RuntimeContext, run_id, session_id	Missing config, invalid paths, dependency failures	Used by every later part
Config loader	Resolve settings deterministically.	Raw config sources	Validated RuntimeConfig	Conflicting values, unsupported env, missing mandatory	Feeds model, storage, and safety layers

				fields	
Logger / tracer	Record structured events.	Runtime events, errors, lifecycle signals	Log lines, trace records	Unstructured messages, missing correlation IDs	Consumed by observability and benchmark harness
State store	Persist session and task state.	State updates, checkpoints	Reloadable state records	Write failures, corruption, schema drift	Consumed by memory, recovery, evaluation
Message envelope manager	Normalize messages.	User input, system instructions, model replies	Canonical message records	Missing roles, unsupported content shape	Foundation for planner and research prompts
Model gateway	Mediate model calls.	Message bundle, runtime config	ModelResponse	Timeouts, empty output, schema mismatch	Base interface for later multi-agent calls
Artifact manager	Own output file creation and indexing.	Generated text, checkpoints, reports	ArtifactRecord	Path collisions, missing permissions, broken manifests	Needed by test, review, deploy, benchmark
Orchestration shell	Host the first executable flow.	TaskRequest, RuntimeContext	TaskResult	Failed transitions, unhandled exceptions	The place where later agents are inserted

Each component should expose one narrow interface. Wide interfaces create accidental coupling, and accidental coupling becomes a maintenance tax that future engineers will pay with interest.

9. Implementation Sequence

Step 1 - Establish the repository skeleton

Create the directory structure, base package entry points, and test folders. This comes first because all later work needs a stable home.

Output: clean importable project structure and a verified run command.

Step 2 - Define the core contracts

Add request, response, state, event, and artifact schemas before any real runtime logic. Contracts come early because they reduce refactoring churn.

Output: canonical data shapes that every component can import.

Step 3 - Build configuration loading and validation

Implement deterministic resolution of environment variables, config files, and defaults. The runtime cannot be trusted until configuration is predictable.

Output: validated RuntimeConfig and clear failure messages.

Step 4 - Add structured logging and correlation IDs

Every later subsystem depends on traceability. Logging is required before stateful behavior is introduced so failures can be explained.

Output: run-level logs with request and session correlation.

Step 5 - Implement runtime state and checkpointing

Add session/task state management after the contracts exist. This stage creates the durable memory of the foundation layer.

Output: reloadable session and task checkpoints.

Step 6 - Add the model gateway

Create the single interface through which the model is invoked. The gateway must sit on top of config, logs, and state, not around them.

Output: one normalized model call path and one normalized response path.

Step 7 - Build the orchestration shell

Wire the minimal LangGraph flow that passes a request through the runtime and model gateway. This is the first end-to-end execution path.

Output: an executable flow from input to output.

Step 8 - Add artifact management

Formalize artifact creation, naming, and indexing once the runtime can already produce outputs. Artifacts belong after the core flow because they record the result of execution.

Output: manifests, checkpoints, and run outputs stored consistently.

Step 9 - Write smoke and contract tests

Protect the new foundation before any later features arrive. Tests are the fence that keeps the base layer from regressing as the system grows.

Output: repeatable verification for boot, persistence, and response flow.

10. Validation Strategy

Validation for Part 1 should prove that the runtime is deterministic, restartable, and traceable. The point is not to prove intelligence. The point is to prove the substrate does not fall apart when later intelligence arrives.

Subsystem	Test type	What is verified	Pass condition
Config loader	Unit + negative tests	Required settings, defaults, invalid combinations	Fails cleanly on invalid input and produces the same config for the same inputs
State store	Integration + restart tests	Session/task persistence and reload behavior	State survives process restart and matches the last checkpoint
Logging	Structured log assertions	Correlation IDs, event structure, error shape	Every lifecycle step emits a structured record
Model gateway	Contract tests with stubbed model	Message bundling, response normalization, timeout handling	Gateway returns a response object that matches the contract
Orchestration shell	End-to-end smoke tests	Input through output control flow	A request can move through the runtime without touching later-stage logic
Artifact manager	Filesystem tests	Path conventions, manifests, collision handling	Artifacts are written to the expected location and indexed correctly

Verification rule

A Part 1 run is only accepted if all of the following are true:
- the runtime boots from a clean checkout
- a task can be created and checkpointed
- the model gateway returns a normalized response
- logs include run_id and session_id
- the session can be reloaded after restart
- artifacts are written and indexed predictably

11. Deliverables

• Repository scaffold with a stable package namespace.

• Configuration system with environment and file-based loading.

• Structured logging and correlation ID generation.

• Core request, response, state, message, and artifact contracts.

• Runtime context and session/task checkpointing.

• Model gateway interface for the base local model.

• Minimal LangGraph orchestration shell.

• Artifact directory conventions and manifest handling.

• Smoke tests and contract tests for the foundation layer.

• Documentation describing how the foundation is started, inspected, and verified.

12. Acceptance Criteria

Criterion	How it is tested	Pass condition
Runtime boots with no hidden manual steps	Fresh environment startup test	The process starts successfully using documented configuration only
Configuration is deterministic	Repeated load test with same inputs	Same inputs always yield the same validated config
State persists across restart	Create session, stop runtime, restart, reload	Reloaded state matches the checkpoint created before shutdown
Every run is traceable	Inspect logs for a known request	Logs contain correlated request, session, and run identifiers
Model invocation follows the contract	Stubbed response test	Gateway returns normalized output and metadata in the expected structure
Artifacts are recorded consistently	Generate output and inspect manifest	Artifact exists, is indexed, and can be referenced by ID
No later-stage subsystem is required	Disable planner/research/memory modules	Foundation still boots and completes its smoke flow

13. Common Failure Modes

Failure mode	Why it is dangerous	Preferred mitigation
Everything stored in one giant runtime object	Makes future features difficult to isolate and test	Split ownership into config, state, logging, artifacts, and orchestration
Configuration read from random places	Creates non-reproducible runs	Use one config loader and one validated config object
Logs without correlation IDs	Makes multi-step debugging nearly impossible	Attach request_id, session_id, and run_id to every event
State mutated without checkpoints	Breaks restartability and recovery	Checkpoint at clear lifecycle boundaries
Model gateway mixed with orchestration logic	Tangles prompt flow with runtime mechanics	Keep gateway and orchestration as separate interfaces
Artifact files written ad hoc	Future review and deployment steps cannot trust outputs	Route all writes through the artifact manager
Premature implementation of later parts	Turns the foundation into a half-finished architecture clone	Keep Part 1 narrow and contract-driven

14. Future Integration

Part 1 is the base that later parts extend rather than replace. The interfaces defined here should be stable enough that future features can plug in without rewriting the runtime.

Next part	How it attaches to Part 1	Why the attachment matters
Part 2 - Filesystem + Local Tooling	Uses the artifact manager, config layer, and runtime context to safely expose file operations.	The runtime needs controlled tools before the agent can manipulate code.
Part 3 - Planning Layer	Consumes request, state, and message contracts to create ordered execution steps.	Planning needs stable task/state models already in place.
Part 4 - Research Layer	Reads from the same message and artifact contracts to keep evidence isolated from core	Research must plug into a clean control plane.

	runtime state.	
Part 5 - Memory System	Builds on session/task checkpointing and persistent state ownership.	Memory needs durable identifiers and stable persistence boundaries.
Later parts	Reuse config, logging, artifact, and state primitives rather than creating duplicates.	The foundation prevents fragmentation across the rest of the system.

15. Completion Checklist

☐ Repository structure exists and is importable.

☐ Core contracts are defined and versioned.

☐ Configuration loading is deterministic and validated.

☐ Structured logging works end-to-end with correlation IDs.

☐ Task/session state can be created, checkpointed, and reloaded.

☐ Model gateway returns normalized responses through one interface.

☐ Minimal orchestration shell executes a complete foundation run.

☐ Artifacts are written through a single manager and indexed.

☐ Smoke tests and contract tests pass in a clean environment.

☐ No planner, research, memory, testing, evaluation, recovery, or graph logic is required for a successful Part 1 run.

Completion definition

Part 1 is complete when the runtime can reliably boot, create a session, load configuration, invoke the model through the gateway, checkpoint state, write artifacts, and pass all smoke checks without depending on any later-stage subsystem.